

INSuRE Fall 2023
Final Report
Protecting Removable Storage
Sponsor: National Security Agency

National Security Agency

Chris DeWeese - Problem Mentor - cjdewee@nsa.gov

The University of Texas at Dallas

Team 2

Tri Nguyen - Student Researcher - txn180008@utdallas.edu
Andrei Racila - Student Researcher - axr180109@utdallas.edu
Shreya Soman - Student Researcher - svs170430@utdallas.edu
Shuo-Ting Hsu - Student Researcher - sxh200065@utdallas.edu
Brian Ricks - Faculty Support - bwr031000@utdallas.edu

December 10, 2023

Table of Contents

1 Project Summary.....	3
1.1 Top-Level Research Project Description.....	3
1.2 Keywords.....	3
1.3 Acronyms.....	3
1.4 Terminology.....	3
1.5 Team Member Responsibilities.....	4
1.6 Budget.....	5
1.7 Key Deliverables.....	6
2 Executive Summary.....	7
3 Motivation.....	8
4 Project Goals.....	9
5 Research Plan.....	10
5.1 Proposed Project Schedule.....	10
5.2 Project Schedule Flowchart.....	11
6 Anticipated Issues and Challenges.....	12
7 Broader Impact.....	13
8 Literature Review.....	14
8.1 Initial Literature Review.....	14
8.2 Additional Literature Review: USB Attack-specific Findings.....	17
9 Final Progress.....	20
9.1 USB Rubber Ducky.....	20
9.2 LBM Virtual Machines.....	21
9.3 Timing Fingerprint Authentication.....	22
9.4 Challenges.....	23
9.5 Future Work.....	24
10 Investigator Biographies.....	25
11 Bibliography.....	27

1 Project Summary

1.1 Top-Level Research Project Description

The goal of this project is to explore ways to secure systems that utilize removable storage mediums. Not only does the project aim to determine the deficiency of security mechanisms available for computers interacting with removable storage devices, but also to design and implement practical solutions, akin in function to network firewalls, that can securely mediate interactions between users and removable storage devices.

1.2 Keywords

Removable storage, kernel drivers, packet filtering, malware, USB, eSATA.

1.3 Acronyms

- **eSATA:** external Serial Advanced Technology Attachment
- **eBPF:** extended Berkeley Packet Filter
- **ID:** Identification
- **IRP:** Input/Output Request Packet
- **LBM:** Linux eBPF Modules
- **OS:** Operating System
- **SD:** Secure Digital
- **SSD:** solid state drive
- **TPM:** Trusted Platform Module
- **USB:** Universal Serial Bus
- **VD:** Virtual Device

1.4 Terminology

- **Kernel Driver:** A software component that operates at the OS's kernel level and facilitates communication between the USB device and the host computer's OS.
- **Device Enumeration:** Device Enumeration is a crucial process that occurs when a USB device is connected to a host computer. It's the initial communication sequence through which the host system identifies and configures the newly attached device.
- **Vulnerability:** A weakness in an information technology system that can be exploited to compromise the operation of that system's unauthorized activities.
- **Threat:** A vulnerability that has been identified by the adversary and has the potential to be carried out as an attack.

- **Attack:** An attack involves the concrete execution of a threat through the specific actions taken by an adversary.
- **Malware:** Malware refers to any software program or code specifically designed to damage, disrupt, or gain unauthorized access to computer systems, networks, or data.
- **BadUSB:** An attack that utilizes a USB device programmed with malicious software to compromise data and operations of the host computer.
- **Outsider Threats:** Outsider Threats refer to USB-based social engineering attacks that intend to deceive users into connecting untrusted devices.
- **Insider threats:** Insider Threats entail the mishandling of these devices due to human error, which can result in significant consequences.
- **Code Injection:** The practice of utilizing USB storage devices as a means to introduce malware into a system.
- **Data Exfiltration:** An instance where a device may transmit or receive sensitive data to or from an unintended application, posing particular concerns when it comes to sensing devices that could potentially be utilized for surveillance on unsuspecting users.
- **Protocol Masquerading:** The tactic of introducing additional and concealed interfaces to the host operating system, such as incorporating a network adapter within an audio headset, for deceptive purposes.
- **Protocol Corruption:** When an attacker takes advantage of kernel-mode arbitrary code execution vulnerabilities during the host's device enumeration process, effectively rendering the physical connection of the device as the sole remaining obstacle to compromise.
- **Signal Eavesdropping:** The ability to intercept sensitive data by physically observing the communication between the host and peripheral, similar to the functionality of keyloggers.
- **Signal Injection:** The utilization of analog signals to transmit sensitive data, resulting in the inadvertent leakage of information to external sources outside the machine.
- **VD:** Virtual devices/workstations that users can ssh into to work remotely.
- **IRP:** IRP or I/O Request Packet, is a data structure used by Windows to communicate with drivers/other devices. It contains details on the type of I/O operation, the device/driver it wants to convey it to, etc.

1.5 Team Member Responsibilities

All team members will participate in research into removable storage medium protocols, analyzing previous works in the topic area, and implementing proposed solutions into code.

Tri Nguyen will serve as chief liaison with the project sponsor. In addition, due to his experience participating in cybersecurity research and internships, he will serve as project manager, ensuring project milestones are met, that team morale remains high, resolving project

blockers, and providing high-level guidance for team members regarding questions or issues they may have regarding the project.

Because of their knowledge of formal verification techniques and experiences with cybersecurity internships and research, **Shreya Soman** will take the lead in creating any formal specifications and protocols needed to implement the project solutions and ensuring their correctness. She will also aid in ensuring any code produced in this project conforms to specifications and is free of any possible security bugs (e.g.: buffer overflows, use-after-free issues, etc.).

Shuo-Ting Hsu and **Andrei Racila** will assume responsibility for leading the code development needed to accomplish the project goals given their experience in developing software. Their experiences in working with drivers and low-level system programming will be valuable for the implementation of the project's security framework.

1.6 Budget

For the project team to learn more about the low-level details of removable storage devices, to understand how attackers utilize removable storage devices to spread within networks, to understand the insecurities of removable storage devices, and to properly test and benchmark the project team's solution against case studies and workloads that mirror real-world conditions (akin to the work done by Tian et al., 2016 on USBFILTER), it is ideal that the team has access to several physical removable storage devices as also suggested by the problem mentor. Specifically, the team requires procurement of a Raspberry Pi Pico module for simulating badUSB, a Raspberry Pi 4 to act as the host computer, a standard external hard drive, and cables for connecting components. The total budget is expected to be within \$200.

In terms of time budget, this project is estimated to consume around 518 hours, accounting for 9 hours per week per student researcher and 1 hour per week of the problem mentor's time.

1.7 Key Deliverables

Key Deliverables	Proposed Due Date
Project Proposal	09/08
Phase One - Literature Review on Specifications	09/22
Phase Two - Development of Rules Language and Framework	10/06
Midterm Presentations	10/13
Midterm Report	10/15
Phase Three - Design and Implementation of Solution	11/17
Documentation	12/01
Final Presentation	12/08
Final Report	12/10

2 Executive Summary

This proposal entails explaining the insecurity of removable storage mediums both from a theoretical perspective and a practical one, as well as implementing concrete solutions to remedy these problems. The project will focus first on analyzing the problem space and generating reports, then will transition to implementing concrete solutions. Previous work in this problem space is analyzed in order to draw inspiration from and to identify areas of improvement applicable to the project's area of interest.

Instead of addressing threats at the Physical Layer, such as Signal Eavesdropping and Signal Injection, our project concentrates on the Transport Layer and Application Layer. This approach aims to provide software solutions to mitigate risks related to Code Injection, Data Exfiltration, Protocol Masquerading, and Protocol Corruption. Our efforts will be focused on both gaining a thorough knowledge base of attacks and defenses relevant to removable storage security, and developing a software solution incorporating that knowledge to further address the current gaps in securing removable storage devices. Tentative budgets and schedules are also elaborated on in this proposal.

3 Motivation

Removable storage medium for computing devices has existed for as long as computers have been around, having evolved from punch cards to floppy disks to now a plethora of different devices such as USB flash drives, SD flash memory cards, portable SSDs, optical disks, and many more. Modern commercial removable storage devices offer large amounts of storage (up to terabytes of memory) and come with many convenient capabilities, such as the ability to reformat a USB flash drive to use any file system a user desires.

These properties, as well as the observation that system users typically utilize the same removable storage devices between different machines, have led to attackers using removable storage devices as a medium to spread malware deep into even hardened and remote computer networks. Despite the fact that malware utilizing removal storage mediums have existed for decades and have infected millions of devices, individuals and organizations today still fall victim to these classes of malware. These issues not only affect individual security, but also national security, since removable storage devices offer an effective and straightforward path for an attacker to gain a foothold into even air-gapped and hardened networks.

Despite there being research on removable storage security and the existence of tools such as USBGuard (<https://usbguard.github.io/>) that users can install on their devices, these tools focus on authenticating certain classes of devices based on attributes such as device ID, rather than examining the actual contents stored on these devices for malicious contents. This project group aims to answer two questions: What explains the lack of widespread security measures in hardware and software that interact with removable storage mediums? And is it possible to develop tools that can act akin to network firewalls (filtering out malicious traffic going from removable storage mediums to user machines based on user-defined rules) that are easy to use and install on a variety of devices while also properly mitigating attack vectors utilizing removable storage devices?

4 Project Goals

The first goal of this project is to analyze a select group of removable storage protocols, starting with USB and eSATA, for security vulnerabilities and to compile a thorough report of our findings. This will involve analyzing breaches in the past of computer networks through the use of removable storage devices and finding common threads among the way attackers utilized these devices. Not only will this involve reading reports and papers from computer security research groups, but it will also ideally entail analyzing and reverse engineering the code of these malware samples at the function level in secure environments to understand what specific functionality of host machines, host operating systems, and removable storage device capabilities were exploited in these attacks.

The other goal of this project is to implement a software solution to these issues. Ideally, it should be easy to install for a wide range of users while not requiring extra hardware to be purchased. First, drawing upon the design of network firewalls and other security solutions, we aim to create specifications for a rules language that can serve as a filter for data entering and leaving removable storage devices. The rules language will have a comprehensive set of features, being able to filter data packets not only by device ID and type, but also by content and other properties. We then will implement the rules language in an operating system through the use of kernel-level drivers as they can be installed on operating systems without the need to recompile the kernels or any other difficult setup procedures. The drivers will also support other features such as tamper resistance and the ability to easily extend support for other removable storage protocols. Our solution will be benchmarked against case studies and their performance and effectiveness will be analyzed. If we are unable to fully realize our specifications into software, we will detail the difficulties we encountered in doing so and lessons learned for future teams to use when analyzing this problem space.

5 Research Plan

5.1 Proposed Project Schedule

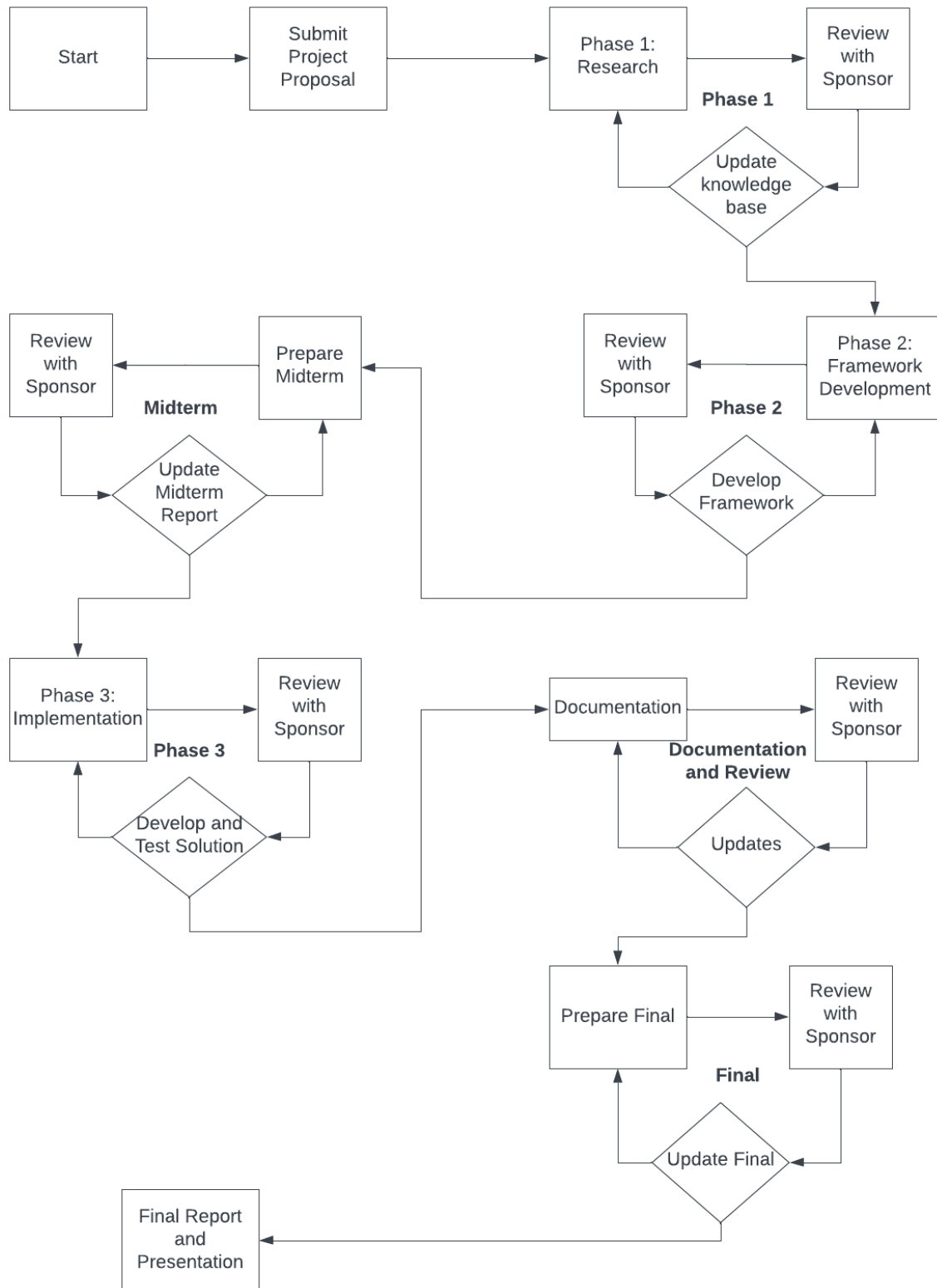
Our research project will consist of three phases. In the first phase, team members will conduct paper reviews on removable storage device security and familiarize themselves with specifications for removable storage medium (USB, eSATA, etc.). This will allow the team to catalog possible exploits and lack of security mechanisms in these specifications. The team will also read and analyze reports concerning malware samples that have utilized removable storage devices to infect machines and exfiltrate data from secure networks. This initial research will help the team determine which security vulnerabilities these malware utilized, and how effective and generalizable, our proposed solutions will be in preventing damage from not only these malware samples, but also potential exploits in the future. These findings will be summarized and discussed at the weekly meetings with the problem mentor to help the team gain a greater understanding of the problem space and a clearer determination of the solution to implement.

The second phase will consist of creating a specification for a rules language and a framework that can be implemented in software. The team will draw from papers, network firewalls, endpoint security platforms, and other computer security solutions to guide the work in this phase. The team will then evaluate the effectiveness of the rules language in dealing with security risks from removable storage devices and verify the correctness of the rules language.

The third phase will be the implementation of the rules language and framework into the Linux and/or Windows operating system as either kernel drivers or directly integrated into the kernels. The approach in this phase will consist of a design period where the team makes design choices, and an implementation period where the team realizes the design into a software product. At the end of this phase, the team hopes to have a working prototype of our approach for further testing and analysis.

Afterward, the team will be conducting a review of the implemented solution. The team will test the software artifact to determine its performance, ease of use, and effectiveness in neutralizing malicious code stemming from compromised USB devices. During this phase, the team hopes to identify any potential drawback of our approach to be addressed in future research, and to resolve any issues or bugs with our implementation. The team anticipates this to be the shortest phase as we expect the bulk of time to be spent on conducting research and implementing a solution. A timeline of the research plan phases can be found in section 1.7. An overall workflow for project phases can be seen in the figure below.

5.2 Project Schedule Flowchart



6 Anticipated Issues and Challenges

Though the members of the project team have conducted research relating to computer security and have extensive software engineering experience, they are less experienced with firmware and device driver programming. Knowledge of several removable storage protocols is needed to conduct this project and the specifications for these protocols are intricate. Obtaining a working understanding of how these protocols work may take up time on the project that could be spent developing new tooling and specifications.

7 Broader Impact

Removable storage devices are ubiquitous among users of computer systems; from individuals backing up important documents from their laptops onto removable SSDs, to large organizations using removable storage mediums to transfer sensitive data between siloed networks. Due to their widespread use and the historical lack of secure software to handle these devices on the part of operating systems, removable storage devices have been utilized by malicious attackers for decades to infiltrate even extremely hardened, air-gapped networks. Even recently, sophisticated threat actors utilized USB removable storage devices to spread malware to target oil and natural gas organizations in Asia (Joven et al., 2023).

The goals of this project are to not only understand how to harden a spectrum of important systems against threats utilizing removable storage mediums, but to also work towards building new types of firewall-esque software that can mitigate threats stemming from removable storage devices while also being easy to use and install by computer users, ranging from individuals to large entities such as government departments.

8 Literature Review

8.1 Initial Literature Review

“Plug & Pray” Today – Understanding USB Insecurity in Versions 1 through C (Tian et al., 2018)

In this article, the author outlines the evolution of USB, the USB protocol, USB Attack Vectors, and mitigations against USB attacks. They discussed two mechanisms relevant to USB composite devices: Common Class Specification introduced to codify different kinds of peripherals, and Device Enumeration used to connect them.

The USB functionality is categorized into abstract communication layers to provide a structured framework for understanding these attacks. The Human Layer is the highest level, involving actions and communications between human stakeholders. Abuse at this layer can be categorized into Outsider Threats and Insider Threats. The common outsider threat was USB-based social engineering which aimed to trick users into plugging in untrusted devices. The insider threat such as mishandling these devices is usually due to human error. The Application Layer represents user-level programs on the host and capabilities on the device. The attacks in this layer typically fall into two categories: code injection, where the attacker uses USB devices to inject malware, and data exfiltration, where the device may send or receive sensitive data to or from an unintended application. The next layer is the Transport Layer which encompasses both device firmware and host OS containing the USB stack. The attacks in this layer can be categorized into Protocol Masquerading which will provide additional, obscured interfaces to the host OS, and Protocol Corruption which the attack can exploit kernel-mode arbitrary code execution vulnerabilities during the host's device enumeration. The last one is the Physical Layer which represents the communication over the USB bus. There are two types of attack in this layer, Signal Eavesdropping and Signal Injection. In conclusion, there are several offensive primitives that are leveraged in USB-based attacks, such as Trust by Default and Attack Transcend Layer.

The paper also mentioned some defenses based on the layer that the attacks target. The paper provides five methods to secure the Human Layer, including Security Training and Host-Based/Device-Based Auditing. For defenses targeting the Application Layer, they focus on the host OS and its drivers. The paper also provides three methods to defend the Application Layer, which are System Hardening, Driver-Based Access Control, and Device-Emulating Honeypots. For the Transport Layer, the suggested defenses are Firmware Verification, USB Stack Fuzzing, USB Packet Firewall, and Host-Emulating Honeypots. Lastly, for defense of the Physical Layer, two methods are mentioned: Anti-Fingerprinting and Secure Channel. A comprehensive solution to USB security will likely require trust anchors on both host and device sides. Secondly, a comprehensive solution should be capable of centralizing context from all

operational layers before making security decisions. Finally, the defenses for signal injection are still missing.

Making USB Great Again with usbfilter (Tian et al., 2016)

This paper focuses on the problem of securing host operating systems from malicious USB devices, encompassing not only USB removable storage but also ex.: USB peripherals, such as keyboards. The authors' threat model consists of attackers who have limited physical access to user machines (i.e. they are able to tamper with USB devices and insert new USB devices onto host machines, but otherwise can't tamper with the actual hardware in the host machines) or full network access who may utilize USB devices to spread between machines on the host network or exfiltrate data to a point outside the network. To protect against this class of attackers, the authors modify a version of the Linux kernel to integrate a piece of software they developed titled USBFILTER. Design goals included tamper resistance (through the use of TPM attestation, Intel Trusted Boot, SELinux policies, and modification of the kernel itself, since if USBFILTER was implemented as a kernel module it could be unloaded), complete mediation of traffic between USB devices and host applications, extensibility, and the ability for users operating in Linux userspace to write formally verified packet filtering rules for USBFILTER to utilize when filtering USB device traffic. After implementing the concept in a version of the Linux kernel, they benchmark the implementation against several case studies (such as preventing data exfiltration from networks onto USB devices and allowing only USB charging for devices) as well as certain workloads. They determine that not only does their concept satisfy their stated design goals, but that their code introduces only a low amount of overhead when communicating with USB devices.

USB Traffic Analyzer - digUSB (Ticu, 2021)

In this article, the author provides a tool, digUSB, that captures detailed traffic between a USB device and the host computer for logging, analysis, and recovery purposes. The tool is capable of filtering the captured traffic packets using various attributes such as the transfer type of the packet, the configurations of the USB, and the data strings being transferred. The tool can also aggregate traffic statistics to provide an analyst with more general patterns of behaviors for each USB device connected to the host computer.

The author tested digUSB on a penetration testing tool called BashBunny capable of various USB-based attacks such as keystroke injection, data exfiltration, and malware installation. digUSB was able to log the malicious keystrokes and a ransomware script sent by BashBunny, indicating that such a tool can be used to aid recovery from a USB-based attack on a computer system.

WMUD: A Whitelist Method of USB Devices Redirection for Virtual Desktop (Xinling Shen et al.)

The article filled in the security demands created with the advent of remote work via VDs, or virtual workstations. This is implemented by creating a portable filter within VDs to screen USB devices. Each USB device has a special 1:1 channel to redirect information, instead of being processed in the “auto” channel. Devices are restricted based on items like vendor ID and prior knowledge. The filters are stored with the launching configurations of the VD on servers. The redirection channels proposed prevent attackers from redirecting information into the VD via USB drivers but don’t feature advanced filtering methods. This shows us the need for a generalized filtration/analysis of “USB-borne” attacks within the industry.

Exploring the Taxonomy of USB-Based Attacks (Mamchenko et al.)

The article analyzed the existing attack taxonomies and proposed a generalized uniform USB-based attack taxonomy. It also clarified the interdependence between attacks, threats, and vulnerabilities by defining vulnerabilities as prerequisites for unauthorized activities, threats as known vulnerabilities with the potential for exploitation, and attacks as the practical realization of threats through specific adversary actions.

In order to create a systemized and uniform taxonomy for USB attack taxonomy, they first analyzed the attack taxonomy, which contains developed cyberattack taxonomies, and decided to add Attack Mechanism, Assets, and Type of Damage from Attack into their new attack taxonomy. They also added several bases including Adversary Intentions, Object of Impact, Level of Security, Level of Complexity, Severity of Consequences, Source of Threat, Impact Layer, and Actions. A comprehensive evaluation of USB-based attacks across all bases simultaneously offered insights into the adversary's capabilities, potential consequences, attack vectors, and possible affiliations, enabling effective security provisioning for target objects and identifying shortcomings in existing protection systems when appropriately managed.

Research and Application of USB Filter Driver Based on Windows Kernel (Shaobo Li et al.)

This paper presented a potential construction of a USB filter driver to enhance access control for USB drivers within Windows kernels. The driver takes advantage of IRP-based communication and intercepts it to determine the type of operation on the USB drive. This allows for the creation of a more fine-grained access control protocol. Compilation of filter drivers and the installation of NT-style drivers are also discussed.

As an example of access-control policies, the authors propose turning a USB device into a read-only device to prevent unauthorized modification and deletion of data. This can also prevent the spread of malware by shutting down write operations. The paper also details the representation of USB drivers in a Winx32 system, including the WDM driver model and the driver stack. It provides a discussion regarding the security of USB storage devices via the constant monitoring of IRP instruction sets to/from the device.

8.2 Additional Literature Review: USB Attack-specific Findings

USB malware implicated in fileless attacks. Trend Micro (Gelera, 2017)

The reported malware uses a script to install a malicious backdoor on the host computer. The infection vector is a USB flash disk with two encrypted malicious files. When a user plugs in the USB and opens a file in the USB, the malicious files are decrypted and loaded into memory. The payload downloads the backdoor program and executes it. It also creates an autostart registry entry so that every time a user logs in to the host computer, the same backdoor is downloaded and executed again, leading to persistence.

The Spies Who Loved You: Infected USB Drives to Steal Secret (Joven & Kiat, 2023)

Mandiant's Report on 2 very common USB Attacks: SOGU and SNOWYDRIVE

SOGU

SOGU Malware is the most prevalent USB-based cyber espionage attack according to the report. It performs data exfiltration on host computers and is used in stealing trade secrets and national secrets. The malware is linked to espionage groups in China.

SOGU Infection happens when a user plugs in an infected USB containing an installer for a legitimate program (AdobeUpdate, CEFHelper,...) along with a malicious DLL loader and a malicious payload (encrypted shellcode). When the user opens the installer, the installation will side-load the DLL, then decrypts and run the payload. When the shellcode is decrypted and run, SOGU first gathers information about the host machines, the processes running on the host, and the host's network connections and configurations. Then, it searches for certain document files (Word Docs, Excel Spreadsheets, and PDFs) on the host system and creates encrypted copies of them. The malware then exfiltrates all the data it gathered to its command and control center over various protocols (HTTP, HTTPS, Custom TCP/UDP protocols...)

For persistence, SOGU creates a hidden directory that looks like one from a legitimately installed software. Then creates a Run registry key so that the malware executable is run every time a user logs in. To spread, SOGU can copy itself onto other USB devices connected to the infected host to propagate between hosts.

SNOWYDRIVE

SNOWYDRIVE is a malware that creates a backdoor for remote access and data exfiltration of host systems. It was found affecting systems in various oil and gas companies in Asia. The backdoor created by SNOWYDRIVE is powerful. It can perform system commands,

allowing remote access to the infected host to list, read, write, execute files, and for exfiltrating data.

SNOWYDRIVE infection of a host computer is through an infected USB containing an executable that serves as a dropper that writes several malware payloads to disk and runs them. The payloads all consist of an executable and a malicious DLL to be side-loaded, similar to SOGU. Each payload serves a different function, which can be: creating the backdoor, establishing persistence (via registry key), infecting another USB flash drive, and configuring the malware and host to evade the host system's security measures.

USBEE: Air-gap covert-channel via electromagnetic emission from USB (Guri et. al., 2016)

This modified USB device manipulates rapid voltage changes associated with the transmission of bits into the USB to generate 240Mhz and 480Mhz signals. Signal can be received using RTL-SDR software-defined radio receiver to collect exfiltrated data. USBee only requires permission to create a file on the USB, no admin permissions. The authors suggested several defense methods: whitelist electronic devices, don't allow RF Receivers. Monitor I/O operations for patterns of USBee, shielding EMR from USB components.

Breaking the silence - recent Truebot activity (Pereira, 2022)

Cisco Talos Intelligence Group's report on TrueBot - a Botnet that spreads through USB

The malware exploits CVE-2022-31199 to execute the command bitsadmin (a command line tool for downloading and uploading files) to install a bot on the host machine and turn the host into a part of a botnet. The exploited vulnerable program is Netwrix Auditor, which typically has extensive privileges over the host device and network connections, allowing the malware a high level of access to carry out domain-wide infections. The initial infection starts with an infected USB device.

A border-hopping PlugX USB Worm takes its act on the road (Gunn, 2023)

Sophos News's Report on PlugX USB worm

Similar to SNOWYDRIVE, PlugX creates a remote access backdoor on host computers by side-loading a malicious DLL through a legitimate executable contained in a USB. It tricks the user into running the executable by creating a shortcut that looks like a drive location. PlugX also searches for Word Docs, Excel Sheets, and PDFs to exfiltrate to its command and control center. Furthermore, it creates a fake Recycle Bin directory and configured the host machine so that deleted files go into its fake Recycle Bin rather than the original host system's Recycle Bin. This fake Recycle Bin serves a location for staging files for exfiltration.

USB Rubber Ducky

USB Rubber Ducky is one way to turn USB devices into a vector for cyber attacks via protocol masquerading. USB Rubber Ducky provides additional interfaces to the host OS and appears on the host as a USB flash drive, but the device actually emulates a keyboard and "type" the "Ducky Script", which allows for capabilities such as searching and running programs (including Terminal, Powershell), and executing scripts with administrator privileges. We will be using Rubber Ducky to simulate BadUSB on our Raspberry Pi Pico

The code for USB Rubber Ducky can be found at :

<https://github.com/dbisu/pico-ducky/tree/main>,

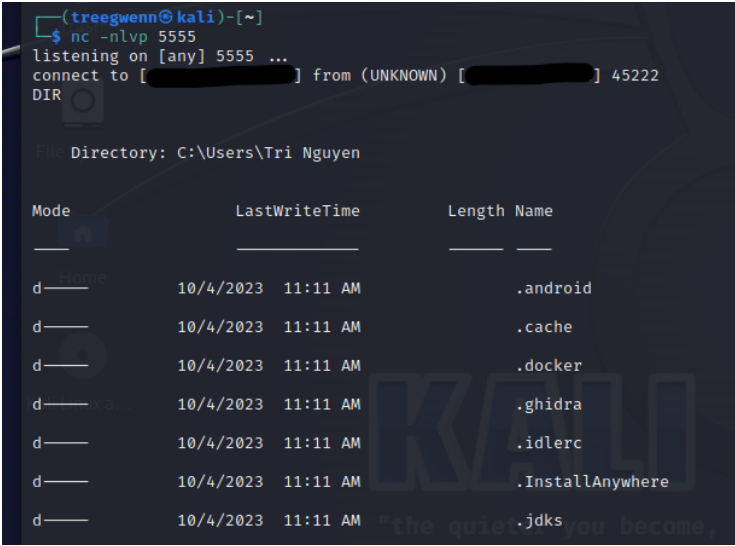
<https://github.com/hak5/usbrubberducky-payloads/tree/master>

9 Final Progress

9.1 USB Rubber Ducky

After a few weeks of research and further literature review (details about additional literature review can be seen in [Section 8.2](#)), we decided, with approval from our sponsor, on a few aspects of removable storage security to focus on for our solution framework; Utilizing eBPF to implement a firewall for removable storage devices, USBGuard to implement an allowlist/blocklist of known malicious peripheral devices, and fingerprinting of the behavior of peripheral devices' behaviors.

We have also emulated popular USB-based attacks on a set of Raspberry Pi Picos using USB Rubber Ducky. Currently, our BadUSB devices can successfully implement keystroke injections and execute payloads to gain remote control by opening a reverse shell and exfiltrating data such as directory information and wifi passwords. On Windows devices, the BadUSB can also establish persistence by writing its backdoor to a script file and adding a startup registry value to run automatically when a user logs in. We are also working on extending its functionality to inject mouse movements like in the USBdriveby attack.



```
(treegwenn@kali)-[~]
$ nc -nlvp 5555
listening on [any] 5555 ...
connect to [REDACTED] from (UNKNOWN) [REDACTED] 45222
DIR
Directory: C:\Users\Tri Nguyen

Mode                LastWriteTime         Length Name
----                -
d-----          10/4/2023   11:11 AM             .android
d-----          10/4/2023   11:11 AM             .cache
d-----          10/4/2023   11:11 AM             .docker
d-----          10/4/2023   11:11 AM             .ghidra
d-----          10/4/2023   11:11 AM             .idlerc
d-----          10/4/2023   11:11 AM             .InstallAnywhere
d-----          10/4/2023   11:11 AM             .jdkcs
```

Figure 1: Reverse shell opened on a Windows 11 target from a USB Rubber Ducky attack

```
xushuoting@Tims-MacBook-Pro ~ % nc -l 8888
[ (root@kali-raspberry-pi)-[/home/kali]
# ls
ls
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
USB
Videos
```

Figure 2: Reverse shell opened on a Linux target from a USB Rubber Ducky attack

9.2 LBM Virtual Machines

We successfully managed to compile and boot the customized Linux kernel that the authors of the LBM paper (Tian et. al., 2019) developed inside a virtual machine. We are currently evaluating where in the Linux kernel the authors added hooks for eBPF calls and determining how to lift out their code that is built-into the kernel itself and into Linux kernel modules, kernel probes, and eBPF programs.

We installed the LBM system on Ubuntu 16.04.7 LTS (Xenial Xerus) virtual machines to observe the interactions between the host kernel and the usb device. We investigated the possibility of lifting out the core functionalities of LBM out of the kernel source, and into more flexible, extendable eBPF programs. Doing so would eliminate the need to recompile the kernel for each installment of the software, allowing for faster and wider deployment, which is crucial for any solution that aims to address the issue of removable storage security on a large scale. Transferring LBM functionalities to eBPF programs can have further benefits, including more runtime security as eBPF programs have additional layers of protections during runtime (such as verifiers, restrictions on modifying kernel memory, etc.) that minimize bugs and vulnerabilities that would otherwise compromise the running of the security system.

After further exploration of the function calls within LBM, we discovered that it is not possible to completely lift its functionalities out of the kernel. Ideally, we want to hook into two critical linux USB handling functions: `usb_hcd_submit_urb()`, which is the first kernel function called when USB packets enter the OS from the USB device, and `usb_hcd_giveback_urb()`, the last kernel function called when USB packets move from the OS to the USB device. Changing the control flow within these functions allows for interception of the processing of USB packets, facilitating deeper analysis of the behaviors of USB devices and making it possible to prevent malicious transfer of data between a host system and a USB device. With eBPF, we

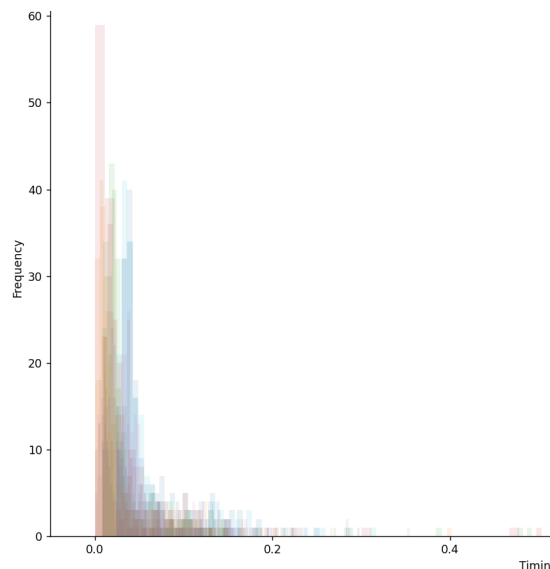
can hook into such function calls, but the only way to change their control flow is through a BPF helper function `bpf_override_return()`, which unfortunately requires the kernel to be compiled with specific configuration options (`CONFIG_BPF_KPROBE_OVERRIDE`), and only on specific functions tagged with `ALLOW_ERROR_INJECTION` in the kernel code. So recompilation of the linux kernel will be required to make full use of LBM's potential. We can still however look to lift other aspects of LBM out of the kernel to maximize the deployability of any solution that makes use of LBM's functionalities.

9.3 Timing Fingerprint Authentication

There are several fingerprint patterns for removable storage. The first one is user input such as entering a password. The next fingerprint pattern utilizes some features in USB devices such as Produce ID, Vendor ID or Serial number. But this information could be forged by an advanced attacker with modified firmware.

The other fingerprint pattern is to measure the time of specific operations such as Handshake and enumeration process or some operations that will interface with the flash controller. However, the handshake and enumeration process can leak information about the host and the behavior of packet transmissions between similar devices do not vary significantly enough to create a unique fingerprint. While the timing of the USB setup packet does not seem to provide unique information to create fingerprints, interfacing with the flash controller can. To create a timing fingerprint, we issued a series of read operations. Each read is randomly chosen to be of size 8KB, 16KB or 32KB, and is to access six logical blocks at six predefined addresses. We set up SCSI Read commands to send to a USB device with some specific flag set to force the system to access the USB drive with each read and disallows the operating system from utilizing cached read data. Finally, we use the CPU clock cycle to measure the time precisely.

Once we've collected this data for each usb device, we compile it into histograms which model the statistical distribution of timing variations in these read commands. These histograms are our fingerprints for each usb device, and look something like this:



This is a timing fingerprint for a Samsung USB device. The y-axis tells us the frequency of a certain read time, and the x axis just details read timings.

This data is then used to train a convolutional neural network (CNN). When a potentially malicious usb device is plugged in, claiming to be something it is not, our CNN will try to see if the device's fingerprint matches up with what the device is saying. If not, we move to blacklist said device and halt the attack.

9.4 Challenges

We use Raspberry Pi Picos to create a USB Rubber Ducky. The challenge is that DuckyScript can only implement keystrokes. If we want to perform more complex actions, we need to run a shell script on the USB device. However, in Linux, the USB device defaults to 'unmount,' which means we can't locate the USB device directory or execute our shell script. We've discovered a way to use DuckScript to create a directory and mount the USB device to that directory. This allows us to access and execute the shell script on our USB device.

In testing out the LBM kernel, we had to first compile and install it. The kernel is based on a now 5 year old Ubuntu Linux kernel and its code repository had little information on how to install the kernel in an Ubuntu environment. We had to rely on Ubuntu documentation to actually build their kernel. Building the kernel took over an hour inside an Ubuntu Linux virtual machine and there was slight difficulty in getting the GRUB bootloader to actually recognize the new kernel. As for the userspace utilities and the rules language associated with the paper, there is little documentation in the code repository to aid in utilizing them. This illustrates not only the challenges with working with the few existing solutions out there, but also potential difficulties in deploying these solutions on a wide-scale.

9.5 Future Work

Broadly speaking, future works in this project can be categorized into two sets of tasks: improving the variety and quality of defenses, and further evaluation and testing of such defenses. In this project, we limited our research and solution to a specific set of defense frameworks and techniques, but the reach and connective power of removable storage devices demands a greater coverage of possible attack surfaces and methods. Future researchers can extend our results into more in-depth analysis of the packets sent between a USB device and its host to create a better model of the behavior of these devices, helping create systems that can distinguish malicious behaviors from benign ones and allowing for more thorough and efficient mitigation of USB-based attacks. They may also look into incorporating more layers of interactions between a peripheral device and its host (physical, human layer, etc.), and applying more robust techniques such as machine learning to create better fingerprints of the behaviors of such devices.

With regards to the evaluation and testing of defenses, future research can look into simulating a greater variety and severity of badUSB attacks, perhaps in the form of a test suite of USB attacks to allow different defense models to be compared using a common baseline. The optimization of the efficiency of these defenses is also a consideration for future research in addition to the security aspect, as we want any defense system to not incur a significant burden on the performance of the host it is running on.

10 Investigator Biographies

Andrei Racila is a Computer Science graduate student focusing on Cyber Security at the University of Texas at Dallas (UT Dallas). They have participated in software development internships at Brookhaven National Laboratory and Amazon where they improved their systems programming skills and learned how to collaborate with other developers to create large software products. Courses they have taken include Information Security, Systems Security and Malicious Code Analysis, Cryptography, Computer Architecture, Machine Learning, and Advanced Operating Systems. They have done research at UT Dallas relating to implementing explainers for graph neural networks (GNNs) and how GNNs can be useful in tracing and detecting the activity of malicious actors in computer systems at the operating system process level. They have competed in competitions relating to computer science such as ACM ICPC, CTFs, and other cybersecurity competitions, such as the 2022 NSA Codebreaker Challenge where they, alongside 4% of participants, achieved solver status. They have experience programming in C, C++, Rust, x86-64, ARM, and Python as they have used them extensively in classes, work, and personal projects.

Shuo-Ting Hsu is a Computer Science graduate student in Cyber Security track. His background is working as an SDE intern at Amazon Web Services to design and implement FIPS support in Redshift Driver such as JDBC, ODBC, and Python Driver. He also has experience working in Trend Micro as a Software Testing Engineer to develop the HNS SDK Automation Testing System. Shuo-Ting has taken courses in Information Security, System Security & Malicious Code Analysis, Network Security, Data and Application Security, Advanced Operating Systems, Advanced Computer Networks, and multiple courses in C/C++ and Java development. Furthermore, he has several certifications including CCNA and CCNP and has experience programming in Python, C# and has participated in several CTFs challenges such as the 2022 NSA Codebreaker Challenge.

Tri Nguyen is a second-year Computer Science graduate student with a Bachelor's in Computer Science from the University of Texas at Dallas. His background is working as a Software Developer with the US Government and Student Researcher with the University of Texas at Dallas. Through these experiences, he has learned much about developing secure and safety-critical operational software for Air Force planes and aircrafts, as well as writing and setting up automated tools for statically detecting and mitigating software vulnerabilities, particularly in C/C++. Tri has taken courses in Computer Networks, Network Security, Data and Application Security, multiple courses in C/C++ development, and is currently taking courses on System Security and on Formal Software Verification using theorem provers. In addition, Tri has experience programming in Python and Java, and has competed in and facilitated multiple CTFs.

Shreya Soman is a Computer Science graduate student specializing in Cyber Security at the University of Texas at Dallas. She has worked with NSF-funded research programs as a

software security intern as an undergraduate, conducting literature reviews, and drafting technical presentations and papers. This summer, she worked with Lawrence Livermore National Lab (LLNL) as a Global Security intern. She implemented Serial Model Bayesian Optimization-based techniques using Machine Learning in Python to improve fault detection and performance within a high-parameter-driven model. Shreya has taken courses in Malicious Code Analysis, Network Security, Machine Learning, and is currently taking Advanced Operating Systems, and is studying Software Verification. She is comfortable using Python, C++ and Java and has competed in multiple CTF challenges, the most notable being the NSA Codebreaker Challenge.

11 Bibliography

- Gelera, B. A. (2017, August 30). USB malware implicated in fileless attacks. Trend Micro. https://www.trendmicro.com/en_us/research/17/h/usb-malware-implicated-fileless-attacks.html
- Gunn, A. (2023, May 6). A border-hopping PlugX USB Worm takes its act on the road. Sophos News. <https://news.sophos.com/en-us/2023/03/09/border-hopping-plugx-usb-worm/>
- Guri, M., Monitz, M., Elovici, Y. (2016, August 30). USBEE: Air-gap covert-channel via electromagnetic emission from USB. arXiv.org. <https://arxiv.org/abs/1608.08397>
- Joven, R., & Kiat, N. C. (2023, July 11). The Spies Who Loved You: Infected USB Drives to Steal Secrets. Mandiant (now part of Google Cloud). <https://www.mandiant.com/resources/blog/infected-usb-steal-secrets>
- Mamchenko, M., & Sabanov, A. (2019, October). Exploring the taxonomy of USB-based attacks. In 2019 Twelfth International Conference "Management of large-scale system development"(MLSD) (pp. 1-4). IEEE.
- Pereira, T. (2022, December 12). Breaking the silence - recent Truebot activity. Cisco Talos Blog. <https://blog.talosintelligence.com/breaking-the-silence-recent-truebot-activity/>
- S. Li, X. Jia, S. Lv and Z. Shao. Research and Application of USB Filter Driver Based on Windows Kernel. 2010 Third International Symposium on Intelligent Information Technology and Security Informatics, Jian, China, 2010. (pp. 438-441). IEEE.
- Tian, D. J., Hernandez, G., Choi, J. I., Frost, V., Johnson, P. C., & Butler, K. R. (2019). LBM: A security framework for peripherals within the linux kernel. 2019 IEEE Symposium on Security and Privacy (SP). <https://doi.org/10.1109/sp.2019.00041>
- Tian, D. J., Scaife, N., Bates, A., Butler, K., & Traynor, P. (2016). Making {USB} Great Again with {USBFILTER}. In *25th USENIX Security Symposium (USENIX Security 16)* (pp. 415-430).
- Tian, J., Scaife, N., Kumar, D., Bailey, M., Bates, A., & Butler, K. (2018, May). SoK:" Plug & Pray" today—understanding USB insecurity in versions 1 through C. In *2018 IEEE Symposium on Security and Privacy (SP)* (pp. 1032-1047). IEEE.
- Ticu, M. (2021, March 25). USB traffic analyzer - digusb. In 2021 12th International Symposium on Advanced Topics in Electrical Engineering (ATEE). IEEE.

X. Shen, N. Li, C. Liu and M. Cai. WMUD: A Whitelist Method of USB Devices Redirection for Virtual Desktop. 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS) (pp. 527-533). IEEE